Chapter 9 exercises

DS09-01 • Leftist rank and meld

Use rank(null)=0. H_1 has root 3 with children 7,9. H_2 has root 4 with left child 8 and no right child. Trace meld, swaps, and ranks at each level. Prove that rank r requires at least $2^r - 1$ nodes. Construct a valid leftist heap with linear height.

DS09-02 • Binomial carries and amortized insertion

Meld binomial heaps of sizes 5 and 3, tracing degree carries and explaining the final single tree. For dedicated insertion that stops when low-degree carrying ends, prove amortized O(1) with root-count potential. Does that proof apply if each insertion scans the complete root list?

DS09-03 • The role of Fibonacci cascading cuts

Suppose decrease-key cuts c nodes, including c−1 ancestors that were already marked. Using $\Phi = t + 2m$, explain how potential pays the actual O(c) work. If only the target is ever cut, where does the Fibonacci degree-size proof fail?

DS09-04 • Pairing operations and one shared potential

Complete both delete-min passes for child roots 8,3,9,5,7, retaining entire subtrees. Link prepends the loser as the winner's first child; pair left-to-right and accumulate right-to-left. Define $\Phi = \sum_v \log_2 s(v)$ using LCRS binary sizes. Derive the exact adjacent-link change for sizes a,b and right suffix R, telescope the first pass separately for odd/even degree, and prove the second-pass and complete O(log n) amortized delete-min bound, including k=0,1 and scaling to pay all O(k) work. Give immediate and amortized insert/meld/decrease costs under the same potential, valid-handle/backlink assumptions, and a total bound from empty heaps. Do not import constant amortized insert from a different analysis.

DS09-05 • Workload selection

An algorithm performs V delete-min operations and E decrease-key operations, without meld. Give abstract total costs using binary and Fibonacci heaps, with assumptions. Why do those formulas not alone determine which runs faster on a machine?

Chapter 9 exercises (continued)

DS09-06 • Direct erase, handle lifetime, and one shared potential

Use the chapter's insertion of v0..v32, removal of v0, and decreases of v18,v26. Erase nonroot v29 without changing its key. Show every cut, cascading mark change, child promotion, degree-table link, final minimum and size. Prove that only h29 becomes invalid. Why is removing an arbitrary equal-key node insufficient? Analyze both phases with the same $\Phi=A(t+2m)$, including a root target, empty heap, and final node. Identify the premise broken by cutting v29 without cascading.

DS09-07 • Keyed carries, subtree movement, and root removal

> All keys are distinct. H1 has roots [9,2]: 9 is a leaf; 2 has child list [4,6], and 4 has sole child 8. H2 has roots [7,3]: 7 is a leaf, and 3 has sole child 5. Use min-heap order, increasing root degrees, decreasing child degrees, and links that prepend the losing root. Link 9 with 7, its result with 3, then its result with 2.
>
> 1. Draw each complete resulting tree. Give its root, degree, child list, and remaining root count; check that each of the eight keys occurs once.
> 2. Delete the minimum. Give root-list order, every tree's keys, total nodes, and new min. Why reverse by degree rather than sort by key?
> 3. Two other valid heaps contain seven and three nodes. Tabulate input roots, carry, and retained roots by degree. Which Bk remain? Why cannot three B1 trees form one binomial tree?
> 4. Does using equal-degree links in a Fibonacci heap imply exactly 2^k nodes in every degree-k tree? Identify the missing structural premise.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

References and attribution

Ellis Horowitz, Sartaj Sahni, Dinesh Mehta. Fundamentals of Data Structures in C++, 2nd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study